Discover Applied Sciences

Research

Prediction and classification of IoT sensor faults using hybrid deep learning model

Adisu Mulu Seba[1] · **Ketema Adere Gemeda**[1] · **Perumalla Janaki Ramulu**[2]

Received: 3 August 2023 / Accepted: 27 November 2023
Published online: 20 January 2024
© The Author(s) 2024 OPEN

Abstract
The quality and reliability of internet of thing (IoT) ecosystems heavily rely on accurate and dependable sensor data. However, resource limited sensors are prone to failure due to various factors like environmental disturbances and electrical noise in which they can produce erroneous and faulty measurements. These can have significant consequences across different domains, including a threat to safety in critical systems. Though many researches have been conducted, the existing literature primarily focuses on fault detection in the sensor data, while fault detection is useful, it is still a reactive approach that identifies the faults after they have occurred, meaning that actions are taken after the fault has already impacted the system, potentially leading to negative consequences. In this study, a proactive approach has been proposed by developing a two-stage solution. In the first stage, a hybrid convolutional neural network-long short term memory (CNN-LSTM) model was trained to forecast sensor measurements based on historical data, while in the second stage, the forecasted measurements were passed to a hybrid convolutional neural network-multi layer perceptron (CNN-MLP) model that has been trained to recognize different types of sensor faults and classify the new measurements accordingly. By passing the forecasted sensor values as input to the classification model and categorizing them as normal, bias, drift, random or poly-drift, anticipated the potential faults before they manifest. The publicly available Intel Lab data raw dataset is used, which has been annotated and fault-injected. For regression, gated recurrent unit (GRU), Long short term memory (LSTM), bidirectional long short term memory (BiLSTM), convolutional neural network-gated recurrent unit (CNN-GRU), convolutional neural network-long short term memory (CNN-LSTM), and convolutional neural network-bidirectional long short term memory (CNN-BiLSTM), were evaluated and compared their performance using root mean squared error (RMSE), mean squared error (MSE) and mean absolute error (MAE) with 2-split time series cross-validation. CNN-LSTM outperformed the other models with a Mean Absolute Error of 2.0957 for a 45 time steps forecast. For the classification task, convolutional neural network (CNN), multi-layer perceptron (MLP), and convolutional neural network-multi layer perceptron (CNN-MLP) evaluated using the metrics accuracy, precision, recall, and F1-score with 5 and tenfold cross-validations. CNN-MLP outperformed the others with accuracy of 96.11% for bias, 99.33% for drift, and 98.61% for random and 98.81% for poly-drift. The average accuracy across the 4 faults is 98.21%, which is a 0.3% increase from the baseline work 97.91%. By adopting a proactive approach to sensor fault prediction and classification, this research aims to enhance the reliability and efficiency of IoT systems, allowing for preventive measures to be taken before faults have a detrimental impact.

✉ Ketema Adere Gemeda, ketema.adere@astu.edu.et | [1]Department of Computer Science and Engineering, School of Electrical Engineering and Computing, Center for Electrical System and Electronics, Adama Science and Technology University, Adama, Ethiopia. [2]Department of Mechanical Engineering, School of Mechanical, Chemical and Materials Engineering, Adama Science and Technology University, Adama, Ethiopia.

Article highlights

- The article introduces a hybrid CNN-LSTM as a better performing regression model compared to GRU, LSTM, BiLSTM, CNN-GRU and CNN-BiLSTM evaluated using MAE
- The article presents a hybrid CNN-MLP as a better performing classification model compared to CNN and MLP in terms of accuracy.
- The article shows one can forecast the values the sensors are going to record in the next hours and with a 98.21% accuracy classify the values as normal or one of the 4 faults hence allowing us to take proactive measures if the values are classified to be one of the 4 faults.

Keywords Internet of things · Sensor faults · Multi-step forecasting · Intel Lab data · Convolutional neural network-long short term memory · Convolutional neural network-multi layer perceptron

1 Introduction

The internet of things (IoT) has emerged as a game-changing technology in recent years, with its ability to connect devices and sensors to the internet. As IoT application continues to evolve, one can expect even more significant changes to come to different industries, revolutionizing the way ones work, communicate, and interact with technology [1]. Sensors are the primary driving force behind the IoT revolution [2, 3]. For example, in manufacturing, IoT-enabled sensors are used to monitor equipment performance, reduce downtime and improving overall equipment effectiveness [4]. In agriculture, IoT-enabled sensor devices are helping farmers to improve crop yield and quality, reduce water usage and prevent crop diseases [5]. The health industry is one of the most significant impacts of these sensor technologies, providing real time remote monitoring of health parameters and management of chronic diseases by using IoT-enabled fitness trackers and smartwatches. Also, IoT-enabled medical devices such as pacemakers and insulin pumps. In transportation sensors in the vehicles are used to collect data on vehicle performance, driver behavior and road conditions among other things to enable smarter and safer driving [6].

Sensor devices are basically the building block of IoT infrastructure, that they are capable of observing physical phenomena, processing and transmitting data, making decisions based on the observations, and taking appropriate reactions [1–3]. IoT sensors are designed to be connected to the internet and provide data to cloud-based applications. They typically use wireless communication protocols such as Wi-Fi, Bluetooth, Zigbee or Z-Wave to transmit data, where it can be analyzed and acted upon [7, 8]. This is what differentiates IoT sensors from normal or stand-alone sensors that are usually not connected to the internet, and typically only provide a local output signal, such as analog or digital signal. IoT sensor is composed of four basic components: sensing unit, processing unit, transceiver unit and a power unit. The individual sensors also exist in different types such as temperature sensors, motion sensors, and security sensors in order to monitor and control temperature, lighting, security and other aspects of the home environment. However, these small devices have obviously limited communication resources such as [9, 10]: limited memory storage, limited energy supply, and limited computing capability. Thus, they are usually prone to failure at any time where the functionality of the given IoT system is also questionable if not their future working status are not predicted.

When a sensor fails toward sensing or transmitting it produces corrupted data or erroneous reading or conflicting information that further the IoT system processes this corrupted data, as a result the overall performance of the system is compromised, making it inaccurate and unreliable sometimes even leading to total failure [11]. For instance, in health care system, if a sensor that monitors a patient's heart rate or blood glucose level fails, it can result in improper medication dosing, leading to adverse health effects [12]. In agriculture, the total failure or incorrect working of a moisture sensor can result in over-or-under-watering of crops, affecting crop growth and yield. In the energy sector, an incorrect working or failure of a temperature sensor in a power plant can cause overheating and equipment damage. In logistic, an incorrect working or failure of a temperature-controlled shipping container can lead to spoilage of perishable goods. In aerospace, a failure or faulty working of sensors responsible for monitoring and controlling critical systems such as flight control, navigation, and engine can lead to catastrophic consequences [13]. Also, in smart cities, a failure or incorrect working of a sensor in a traffic management system such as traffic light can lead to traffic accidents and congestion [14]. In oil and gas pipelines, a failure or incorrect working of sensors that monitor the pressure and temperature of the pipelines could cause a leakage which can be an environmental disaster. In summary, sensor failures or incorrect workings have severe

consequences in different industries, ranging from damage to property, system downtime, increased maintenance cost to loss of revenue. This leads in which sensor fault prediction is an important area of research.

Many existing studies have suggested different approaches such as model-based, signature based and data-driven approaches for detecting sensor faults and potential failures. The model-based approach uses a mathematical model of the system to estimate the true value of a sensor reading and detect sensor faults by comparing the predicted and measured values [15, 16]. The signature-based approach compares the current sensor reading with a pre-defined threshold value. If the reading exceeds the threshold, it's an indicative of a fault [17]. The data-driven approach analyzes the statistical properties of the sensor data to learn the underlying patterns and relationships between the sensor reading and the occurrence of faults [18].

Though many works have been done to identify and address these faults throughout the literature, they are mainly focused on fault detection rather than predictions, for example, (Huang et al., [19], using time frequency analysis and deep learning detected and classified 3 types of faults. Uppal et al., [20], by analyzing the existing data (historical data) using machine learning techniques performed binary classification of faults i.e., healthy or unhealthy. Jana et al., [21], by using Convolutional Neural Network (CNN) and Convolutional Autoencoder (CAE) performed detection and classification of 4 types of faults. While detecting a fault is useful, it only addressed the problem after it has occurred. Further, the existing works are controlled with the number of faults (i.e., only specific faults) considered in the literature whereas impossible to be generalized to other faults. With regard to predicting multiple faults, very limited studies are conducted with demanding for performance improvement (such as accuracy). As a result, we are motivated to define the present work to apply in the real world problem. Our proposed system predicts potential faults or anomalies before they occurred by considering the sensor's historical data.

The proposed architecture has two models that work in two phases such as forecasting model that is trained on the historical measurements of the sensors, it is used to forecast the N future values of each sensor. In the second phase, a classification model that is trained on the fault injected dataset, it is used to classify the forecasted values. The fault injected dataset has 4 classes for the 4 fault types and a 5th class to represent the normal (clean) values. So, in the second phase, a multi class (5 classes, i.e., normal, bias, drift, Polydrift and random) classification task is performed to classify the forecasted values.

Commonly, hybrid deep learning models were widely accepted in the recent works in order to achieve better performance [22, 23] since they are able capture the uncertainty and variability in the data and provide more accurate and trustworthy predictions. By ranking and choosing features that scored the highest in the provided data set, only the most pertinent features were picked and utilized in this regard. Such models are also the capability to leverage the benefits of different network architectures and learning paradigms in order to achieve better results when compared with single models. Thus, experiment findings demonstrate that the proposed CNN-LSTM outperformed the other models with a MAE of 2.0957 for a 45 time steps forecasting, and CNN-MLP achieved an average accuracy across the four faults 98.21% when compared with the others.

Accordingly, the main contribution of the study is the development of a multi-stage approach for sensor fault prediction and classification using hybrid deep learning models. As such, a comparative analysis of GRU, LSTM, BiLSTM, CNN-GRU, CNN-LSTM, CNN-BiLSTM for multi-step univariate timeseries forecasting was conducted through analyzing the impact of feature engineering on the predictive performance of timeseries models. Accordingly, since a proactive maintenance and repair can be carried out by identifying the potential sensor faults before they actually occurred, the proposed hybrid deep learning model improved the IoT ecosystems reliability and performances by reducing downtime and costs.

2 Related works

Various works aim to address issues related to sensor failures in order to make the whole IoT application effective. There are several approaches for faults categorization; however, the two major conceptualizations are Data-centric and System-centric *view* to classify the faults from the system point of view and the data point of view. *Data-centric view* focuses on the sensor data itself and classifies faults. This considers each sensor output as an independent data stream and looks for anomalies in the data [24]. This is useful for detecting faults that affect the accuracy of the sensor data. *System-centric view* takes a broader perspective and considers the system as a whole, including all the components that interact with the sensor. This considers faults that arise from the physical sensors themselves and their interaction between different

components in the system. This view is useful for detecting faults that can affect the reliability or availability of the system [24].

Numerous researches have been conducted to address the issue of sensor faults using the approaches such as model-based, data-driven, signature-based, statistical or by combining two or more of them [25–28]. Manzoni et al., [12] employed model-based fault detection and detected Continuous Glucose Monitoring (CGM) sensor's fault in an artificial pancreas. Kalman predictor was used to predict the expected blood glucose level in the normal conditions, accounting also for the meals, carbohydrate contents, and insulin injected by the CSII pump. This predicted value was compared against the measured values by the CGM. Large inconsistencies between the measured CGM values and predicted values of the blood glucose level suggested the presence of faults. Jihani et al., [29] employed a parity space approach to detect and isolate faults in WSN. A mathematical model was developed based on the instantaneous redundant measurement of two sensor nodes planted in the soil with the same depth. A residual signal (parity vector) was generated based on the mathematical model.

Developed mathematical model used to compare the measured sensor data with the predicted values from the model [15, 16]. If there is a significant difference between the predicted and measured values, it is indicative of a sensor fault. This approach requires a priori knowledge of the system and its behavior to construct the mathematical model, and are often more accurate when the model is well defined. The model typically consists of set of equations or rules that capture the dynamics of the system and its interactions with the environment. Hashimoto et al. [30], proposed a multimodal approach to fault detection and diagnosis of internal sensor for mobile robot. Three failure modes (hard, noise and scale failure) of the sensor are handled. The mode probabilities, which are estimated based on a bank of Kalman filters, provide the fault decision which is made by comparing the model conditional estimates in the sensor gain. He et al., [31] proposed a model-based method by considering both disturbances and uncertainties, for detecting optical fiber sensor fault in aero-engine linear discrete time invariant (LDTI) system. The performance was demonstrated by applying the design procedure to a non-linear gas turbine model.

Yan et al., [32] introduced a novel model to for detecting minor soft faults of sensors in an air conditioning system. The authors used kernel principal component analysis for feature extraction and dimensionality reduction, and then the data was passed to a double layer bidirectional long-short term (DL-BiLSTM). The residual which was generated by comparing the output of the DL-BiLSTM with the actual value from the supply air temperature sensor was used to detect the minor faults of the sensors. According to the authors their experiment result showed that their method (KPCA-DL-BiLSTM) was 43% higher than that of KPCA and 18.33% higher than LSTM under 10% drift deviation fault.

Alwan et al., [33] proposed a time-series clustering technique to address the limitations of predictive analysis in detecting long-segmental faults in sensor nodes in large scale cyber physical systems. They also compared feature-based time-series clustering and shape-based time-series clustering and found that feature-based time-series clustering was more efficient long-segmental outliers detection mechanisms. Zhao et al., [34] hired sliding window approach to detect two incipient faults, i.e., constant bias and precision degradation, in industrial processes. The singular value of each window was calculated. The control limits were determined by empirical method using the mahalanobis distance, where each singular value corresponds to a control limit. Uppal et al., [35] developed an early fault prediction model for an IoT environment that uses IoT-based sensors, cloud and ML algorithms. The model was evaluated against four algorithms, i.e., decision tree, k-nearest neighbor, Gaussian Naïve Bayes and random forest. According to the authors, Random Forest showed the highest performance on the given dataset, with a classification accuracy of 94.25%. Liu et al., [36] proposed a deep learning approach for sensor fault self-diagnosing scheme for a wind turbine blade. First a sensor data prediction model was built by mining the inherent relevance between the sensors, and then the residual between the predicted value and the actual value was compared against the control limit to detect the presence of faults. According to the authors, the experimental results of the model showed a good prediction (RMSE = 0.001154, MAE = 0.000214, and R2 = 0.999993) and fault diagnosis performance (a recall of up to 98.43%, an accuracy of up to 98.58%, and a precision of up to 90.01%). Wahid et al., [37] proposed a predictive maintenance based on CNN and long short-term memory (LSTM) for machine failures in industrial plants and factories. LSTM was used to analyze the relationships among the different timeseries data and 1D CNN was used for the effective extraction of high-level features. The authors experimented with CNN, LSTM and CNN-LSTM models for the prediction and found that CNN-LSTM provided the most reliable and highest prediction accuracy over the others. Uppal et al., [20] proposed architecture for monitoring each and every office appliance connected via IoT using machine learning. Data from the appliances is collected by IoT and is sent to the ML algorithms. The ML algorithm performs the classification process of the faults. Safavi et al., [38] recommended a method for predicting the health status of the electronics sensors of autonomous vehicles. For fault identification and isolation, a separate feature extraction block was used and to identify the type of fault and recognize the faulty sensors multi-class

DNN was used. From these works, it has been observed that, each model and study had limitations in their studies interms of many considerations. In Table 1, some of the gaps indicated from the previous studies explained above.

3 Research methodology

This work is aimed to anticipate potential sensor data faults by developing a 2-step hybrid deep learning models for the prediction and classification of sensor faults. An effective and robust solution was generated to predict and classify sensor faults with high accuracy. All the necessary preprocessing and feature-engineering was performed on the data before using it for training. Finally, compared this work against a baseline work and proved that this solution has improved accuracy. The detailed process design induced to this process is described in the Fig. 1.

3.1 Dataset description

Wang et al., [40] using the Intel Berkeley Research Lab dataset (nodes 1,2,33,35 and 37) injected artificial outliers to the normal data to train and evaluated their proposed framework. The outliers consisted 3% of the data and the distribution was random. Emperuman & Chandrasekaran [41] also introduced bias, drift, random and spike artificial faults to the raw data using the algorithm described in [42]. Mishra & Mohanty [43] injected intermittent fault signals to the intel lab temperature and light dataset to test the performance of various machine learning classifiers for intermittent fault diagnosis.

The benchmark datasets used in this paper [42]. The authors also presented a standardized approach (methods and algorithms) to annotate benchmarks and inject faults into clean datasets based on a generic fault model to address the issues of inconsistencies between datasets that is caused by researches having to annotate the datasets by them in order to obtain ground truth. The dataset used in this study contains 10 sensors from the intel lab (temperature). Each sensor has a clean set annotated with the ground truths and 4 sets injected with 4 artificial faults they surveyed from the literature, based on domain expertise and algorithmic methods described in their paper. The dataset is a timeseries data with features timestamp, mote_id, temperature, and has_fault_type. In this regard, the has_fault_type feature is encoded as 0 for no fault, 1 for random, 4 for bias, 8 for drift, and 16 for poly drift.

3.2 Data preprocessing

The data preprocessing steps taken in this research are summarized as follows.

3.2.1 Data integration

Data from multiple sources are combined into a single data frame to obtain a comprehensive dataset. This is done because our data is collected from different sensors, and so the data from different sensors are integrated together to train a model.

3.2.2 Data transformation

Converting the data into a suitable format for analysis. It may include normalizing, scaling, or converting data types. In this work, MinMaxScaler used to transform the data. Also the data type of the timestamp feature was converted from object to datetime format.

3.2.3 Imbalance handling

The dataset was initially highly imbalanced with the normal class significantly dominating the fault classes. Highly imbalanced dataset can lead to biased model during training which can result in poor predictive performance especially on the minority classes for which under-sampling, over-sampling and SMOTE are to balance the dataset.

Table 1 Comparison of previously proposed schemes in terms of technique, performance measures, and limitations

Method(s) implemented	Performance measures	Gaps identified	Reference
One step Kalman predictor with correlogram test and residual punctual value analysis	Residual value analysis showed good performance in terms of sensitivity and FP/day, while correlogram test, although weak in short and minor amplitude fault, is better in longer and deeper faults	One fault considered No comparison made with other methods	[12]
Parity space approach with direct redundancy	Injected faults were immediately detected and isolated	Only abrupt fault tested	[29]
KPCA and double layer Bi-LSTM	10% injection = 97.33%ACC, 99.00% Recall, 97.07% precision. 20% injection = 96.67% ACC, 99.5% Recall,95.67% precision	Only drift faults considered	[32]
DTW, K-Shape timeseries clustering and characteristic based timeseries clustering	Feature-based techniques more efficient than shape-based techniques for long-segmental fault detection	Performance evaluated against shorter time-series window	[33]
Sliding window-based algorithm	95.10% and 95.0% using numerical evaluation and 98.57% and 98.50% using CSTR process for bias and degradation respectively	Only 2 faults considered	[34]
DT, k-nearest neighbor, Gaussian NB and RF	RF performed best with accuracy of 94.25%	Binary classification Result Could be improved	[35]
RF, BP and CNN	Prediction: RMSE = 0.001154, MAE = 0.000214, and R2 = 0.999993, and Diagnosis: Recall 98.43%, Accuracy 98.58%, and a precision of 90.01%	Binary classification Prediction done for 3timesteps	[36]
1D CNN, multi-class DNN	Deep learning outperformed SVM	Lacks generalizability i.e., 3 sensors and 3 fault types	[38]
SVM for detection and identification and predictive algorithm for prediction	Detection and identification accuracies of 94.94% and 97.01%, respectively and prediction accuracy 75.35%	result could be improved using dl methods	[39]

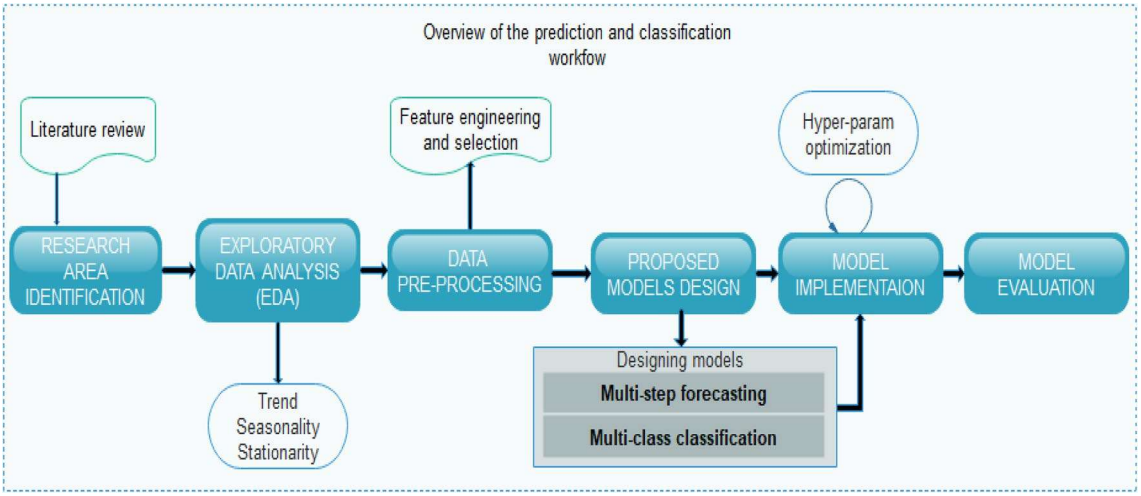


Fig. 1 Research design methodology overview

3.2.4 Stationarity

The data initially contained an increasing trend. Differencing operation was performed in order to remove the trend. Then adfuller test was then conducted to check the stationarity of the data. The test returned a p-value of 0.03 which is less than 0.05 implying the data is stationary.

3.2.5 Feature engineering and selection

3 main tasks were performed under feature-engineering. The first was generation of time-based and rolling window features to capture the local pattern to enhance the predictive power of the model. The second was, among the generated features, selecting the features that contributed more and were more significant than others. The third task was converting the timeseries problem into a supervised learning problem by using the keras TimeseriesGenerator API [43].

Figure 2 show the data preprocessing done for the first stage, i.e., forecasting. After performing timeseries decomposition and detrending the data by performing the differencing operation, preprocessing is then done on the data, i.e., integrating the data from the sensors into 1 dataframe, resampling the frequency to 2 min, converting the timestamp feature to datetime object and so on. Finally, the timeseries data converted into a supervised learning problem by using the Keras TimeSeriesGenerator API.

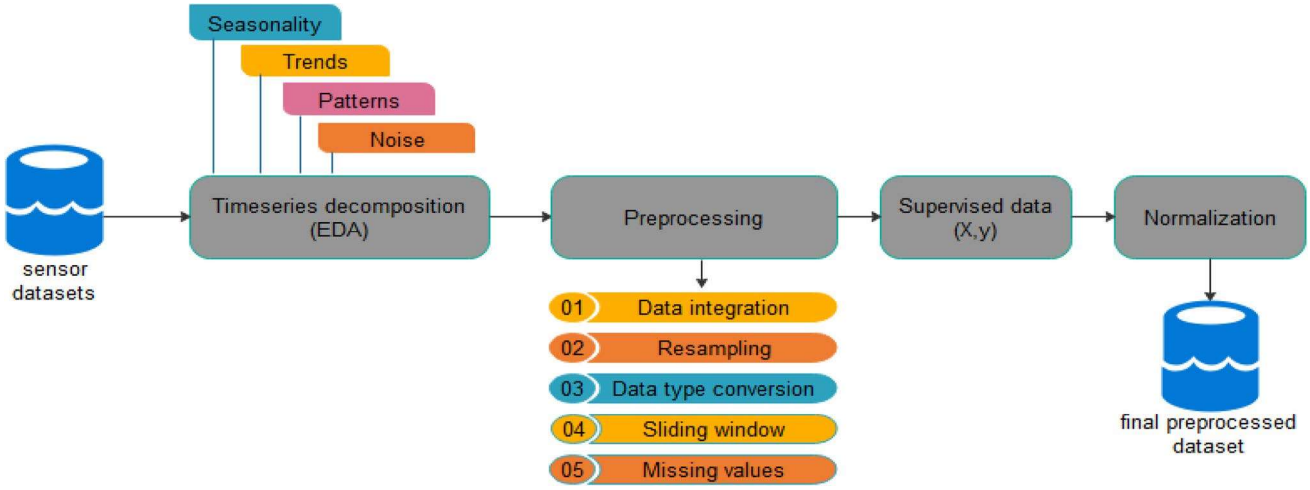


Fig. 2 Data preprocessing for univariate timeseries forecasting

Figure 3 show the similar preprocessing done for the second stage i.e., classification. The main difference being in this case the problem is already a supervised problem as inputs and output are considered, where the output is the has_fault_type feature.

3.3 Model selection

Since sensors are sensing and storing timeseries data, the ability to capture the temporal dependencies is the major criteria for the model selection. The ability of the model to generalize well on unseen data is the second criteria. Based on these criteria, the following candidate models were identified and evaluated each of them.

For regression: the candidate models for regression are GRU, LSTM, BiLSTM, CNN-GRU, CNN-LSTM and CNN-BiLSTM because these models have the ability to capture the temporal dependency between the data and also capture the long-term dependencies that may exist.

For classification: the candidate models for the classification task are MLP, CNN and CNN-MLP.

3.4 Proposed solution architecture

Figure 4 shows the architecture of the proposed system. The architecture consists of a 2-step process, in which, in the first stage a CNN-LSTM model is trained on the historical sensor data. This model is then used to forecast future sensor measurements based on the historical data. In the second stage the forecasted measurements are then passed to the second model, CNN-MLP, which has been trained on fault-injected data to identify 4 types of faults, to categorize the measurements accordingly.

3.4.1 Multi-step forecasting

Multi-step forecasting is the forecasting of many timesteps in the future. There are generally two approaches to forecast many timesteps into the future. The direct multi-step forecast which involves forecasting all the future timesteps at once. The second approach is the recursive multi-step forecasting, whereby a single timestep is forecasted, then the value at the single timestep is taken as input and the next single step is forecasted. This loop continues until the desired number of future timesteps are achieved. The main disadvantage of direct multi-step forecast is that since all future timesteps are forecasted at once, intermediary events or changes are not taken into account. In recursive multi-step forecast, since each forecasted timestep is taken as input for the next timestep, intermediate changes are fed into the model for the forecast of the next timestep. This approach involves predicting the next value in the time series and then using that predicted value as an input to predict the next future time step. This process is repeated until the desired number of future values has been predicted [45, 46].

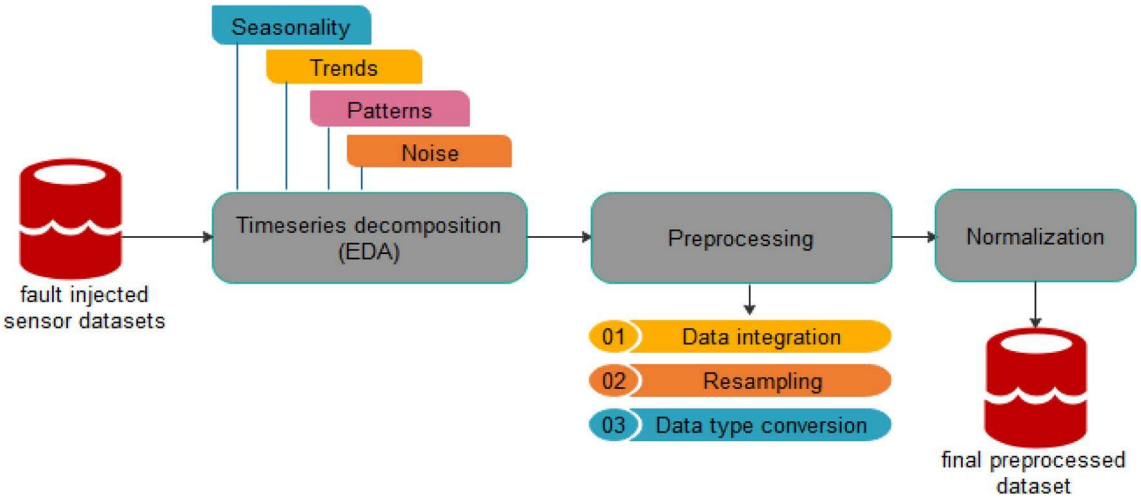


Fig. 3 Data preprocessing for multi-class classification

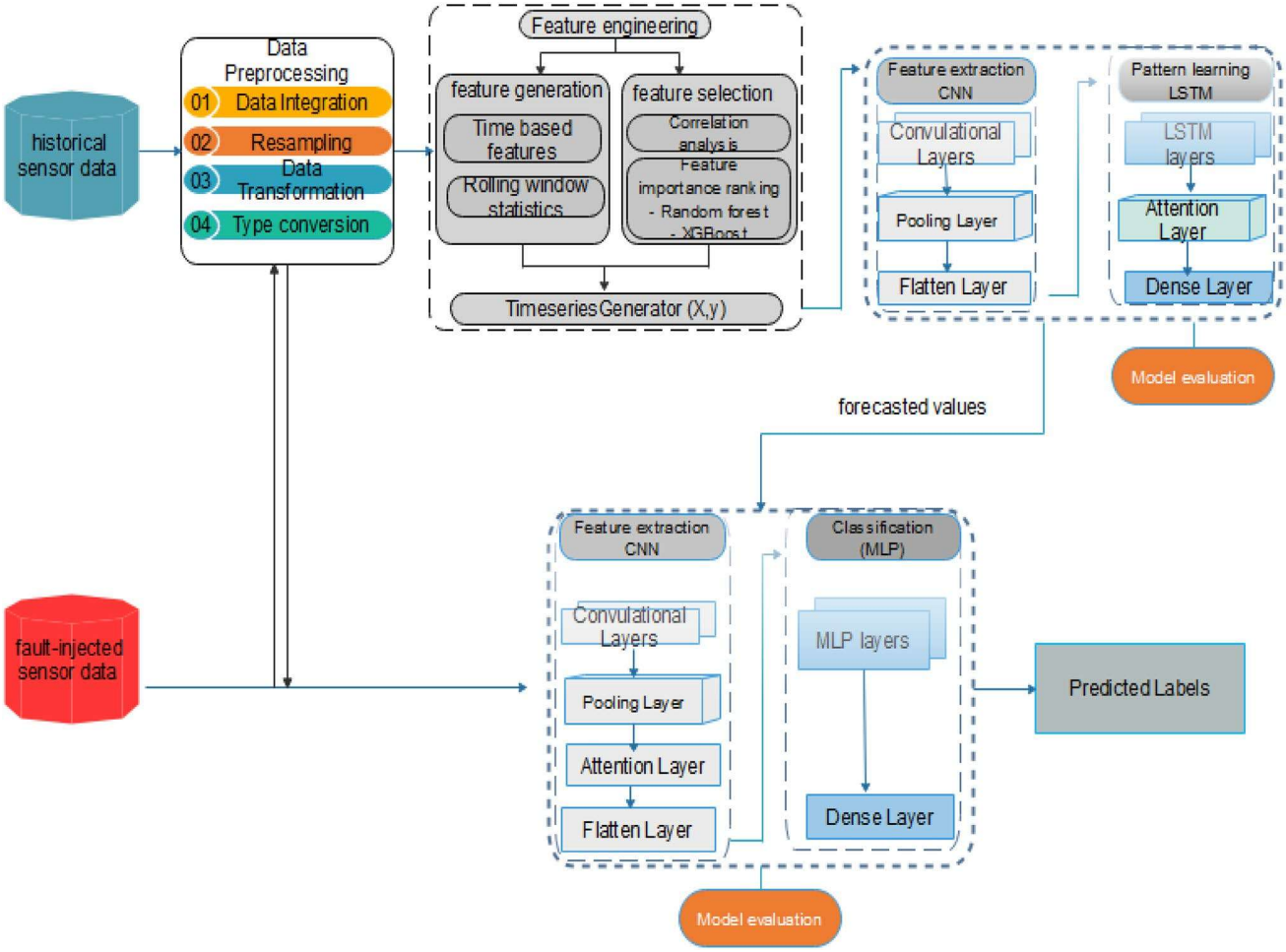


Fig. 4 The Proposed architecture for fault prediction and classification

In this study, recursive multi-step forecast was implemented where the next 45 timesteps were forecasted. According to our design, the next 45 timesteps represent the measurements for the next 1 h and 30 min. CNN-LSTM model was used to perform the forecasting. The reason for combining both of them is to use the strength of each model. The CNN module is used for feature extraction while LSTM is used for learning the temporal pattern and capture the long-term dependency in the data.

3.4.2 Multi-class classification

There are two main approaches to multi-class classification. Direct /native multi-class classification and binary transformation (class binarization). In direct multi-class classification, the classification algorithm is designed to handle multiple classes natively (directly), without any explicit transformation or modification. The algorithm assigns a single class label to each input instance directly. Some algorithms that inherently support multi-class classification include decision trees, random forests, k-nearest neighbors (KNN), and naive Bayes. Another common example of a direct multi-class classification algorithm is the Softmax regression (also known as multinomial logistic regression) which extends the logistic regression to handle multiple classes by using the Softmax function. Class binarization, also known as binary transformation, involves transforming a multi-class classification problem into multiple binary classification subproblems. Instead of directly predicting the multi-class label, a set of binary classifiers is trained to distinguish between each class and the rest of the classes. There are two common strategies for class binarization: one-vs-all (OvA) and one-vs-one (OvO).

For the multi-class classification, because the instances in the data are dependent on each other and it needs clear distinction between the classes, in this study, the binary transformation approach was followed where 4 binary classifiers

were trained using CNN-MLP to distinguish between the 4 faults. During classification of new measurements, the classifier with the highest probability is selected as the predicted class.

Although CNN is mostly popular in computer vision and image processing tasks, it has also shown great potential in handling time-series data [47]. The major difference is that computer vision uses image matrix while time-series uses 1D array [48]. So CNN can be adapted for tabular data by using 1D convolutional layer. As a result, we consider the CNN model that can be combined with the other models as discussed below in order to effectively sensing and storing time-series data as well as effectively model generalizing on unseen data.

3.4.3 Hybrid CNN-LSTM

It is a deep learning architecture that combines convolutional neural networks (CNNs) and long short-term memory (LSTM) networks. The architecture is suitable for processing both spatial and temporal data, making it particularly effective for time-series forecasting problems. The model basically works in 3 main steps.

1. The input time series data is first fed into a CNN layer that extracts the important features from the data. The output of this layer is a set of high-level feature maps.
2. The output of the CNN layer is then fed into an LSTM layer that learns the temporal dependencies in the data. It takes the feature maps as input and produces a sequence of output vectors.
3. The output vectors of the LSTM layer are then fed into a fully connected layer that generates the final forecasts.

The architecture allows the model to learn both the spatial and temporal dependencies in the time series data. The CNN layer extracts the spatial features from the input data, while the LSTM layer captures the temporal dependencies in the data. The model is trained using the mean squared error (MSE) loss function. During training, the model learns to minimize the MSE between the predicted and actual values. To further improved the model, attention layer is added to the model and hyperparameter tuning is also performed in our case.

3.4.4 Hybrid CNN-MLP

This is also a deep learning architecture that combines convolutional neural network (CNN) with multilayer perceptron (MLP). In the architecture, first the input data is fed into a CNN layer that extracts important features from the data. The output of this layer is a set of high-level feature maps. The output of the CNN layer is then flattened and fed into an MLP layer that produces a set of output probabilities for each class. The class with the highest probability output by the MLP is then determined to be the predicted class. The architecture allows the model to learn both the spatial and semantic features in the input data. Four instances of the model are trained on the four sets of data and the instance that produces the highest probability determines the class. The instances (classifiers) are trained using binary_crossentropy as the loss function.

4 Performance evaluations and discussion

For evaluating the performance of the propose models, forecasting analysis and classification analysis metrics were employed. In forecasting, the goal is to predict a continuous variable or a time-series value by analyzing the past values of the time-series [28, 49, 50]. Some of the commonly used metrics in forecasting the model performance are: Mean Square Error (MSE), Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) in which we have also applied. On the other hand, the goal of classification is to predict the class or category of a given instance based on its features or attributes. In this regard, metrics such as accuracy, precision, recall, and F1 score are used in our case.

In the first experiment, after identifying candidate models that are suitable for multi-step forecasting sensor values, this work has evaluated each of them using regression metrics RMSE, MSE and MAE. This experiment has been divided into two sub-experiments.

Experiment (a): Under this experiment 3 models i.e., GRU, LSTM and BiLSTM have been evaluated and tested using the TimeSeries cross validation with 2 splits.

Table 2 Performance of forecasting models

Model	n_splits = 2, n_future_timesteps = 45		
	RMSE	MSE	MAE
GRU	3.6353	13.2154	3.4260
LSTM	2.4569	8.4576	2.3862
BiLSTM	2.3558	7.0005	2.3324
CNN-GRU	2.2061	6.0298	2.1585
CNN-LSTM	2.1037	5.7972	2.0957
CNN-BiLSTM	2.2522	6.4196	2.2442

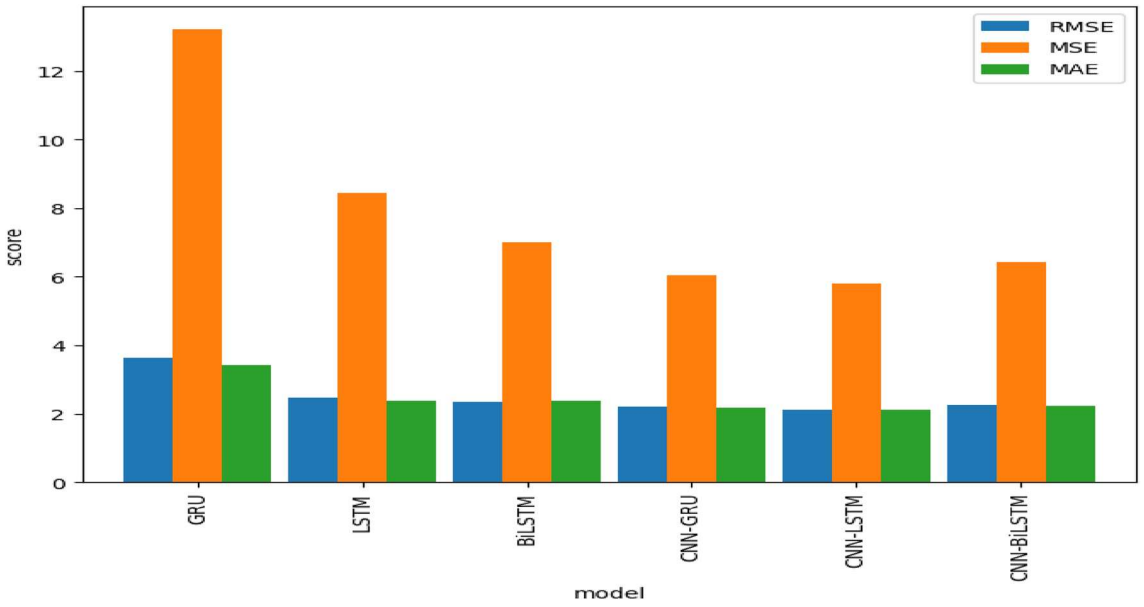


Fig. 5 Forecasting performance evaluation using plot

Experiment (b): Under this experiment, to leverage the strength of CNN model to extract features, the CNN architecture has been incorporated into the above 3 models (*exp a)* and designed 3 hybrid models CNN-GRU, CNN-LSTM and CNN-BiLSTM and evaluated the performance of each of them again using the TimeSeries cross validation with 2 splits.

In the first part of the experiment, BiLSTM performed better than the others with a MAE of 2.3324 forecasted over 45 timesteps, and LSTM came in second with MAE of 2.3862. In the second part of the experiment, after the hybridization, all of the 3 models showed improvements, with CNN-LSTM showing the highest improvement of MAE 2.0957 and CNN-BiLSTM following up with 2.2442 (Table 2).

Figure 5 shows the graphical representation of the performances of the identified six regression model as evaluated with RMSE, MSE and MAE.

Figure 6 shows the training and validation loss curve of the CNN-LSTM model. The model was trained with the mse loss function and an early stopping of 10 patience. The graph shows that the model does not overfit or underfit and converges between epochs 50 to 60.

In the second experiment, three candidate models have been identified (i.e.,CNN, MLP and CNN-MLP) for sensor fault classification task and evaluated their performance using evaluation metrics accuracy, precision, recall and f1. We also employed cross validation in order to avoid overfitting as well as assessing the performance of our model in generalizing the new data that it has not seen before. In this regard, 5 and 10 are the most commonly selected number of folds that are used in the research for grouping the data, i.e., one group as a test set and the rest as a training set. Since binary transformation is considered for multi-class problem, for each fault a classifier is trained and the classifier with the highest probability is selected.

The same setting is conducted for all of the selected three models using 5-and 10-folds cross validation in which their perforances were reported in Table 3 and 4 respectively. Accordingly, the CNN-MLP outperformed the other two models

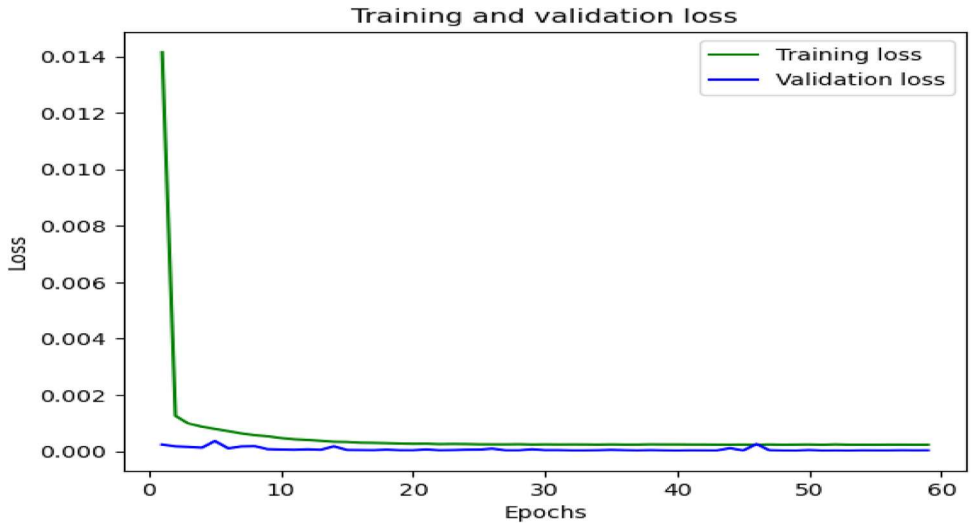


Fig. 6 Training and validation loss CNN-LSTM

Table 3 K5k validation results

Model	Classifiers	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
MLP	Bias	94.70	94.90	94.70	94.77
	Drift	99.00	99.10	99.09	99.09
	Polydrift	98.80	98.20	98.11	98.11
	Random	98.45	98.44	98.43	98.43
CNN	Bias	96.00	96.10	96.05	96.05
	Drift	99.10	99.20	99.19	99.19
	Polydrift	98.60	98.60	98.61	98.61
	Random	98.50	98.54	98.50	98.50
CNN-MLP	Bias	96.10	96.40	96.11	96.09
	Drift	99.30	99.30	99.30	99.30
	Polydrift	98.80	98.81	98.80	98.80
	Random	98.60	98.62	98.60	98.60

Table 4 K10k cross-validation

Model	Classifiers	Accuracy(%)	Precision(%)	Recall (%)	F1 (%)
MLP	Bias	94.78	94.95	94.78	94.77
	Drift	99.03	99.17	99.16	99.16
	Polydrift	98.81	98.83	98.81	98.81
	Random	98.50	98.49	98.48	98.48
CNN	Bias	96.05	96.16	96.10	96.10
	Drift	99.13	99.30	99.29	99.29
	Polydrift	98.63	98.63	98.64	98.64
	Random	98.54	98.58	98.54	98.58
CNN-MLP	Bias	96.11	96.42	96.12	96.10
	Drift	99.33	99.33	99.33	99.33
	Polydrift	98.81	98.83	98.81	98.81
	Random	98.61	98.65	98.61	98.61

when one compares its accuracy results under both cross validations. Thus, the reported accuracy of CNN-MLP would be 96.10 for bias, 99.30 for drift, 98.80 for poly-drift, and 98.60 for random under 5-folds cross validation, whereas 96.11% for bias, 99.33% for drift, 98.81% for poly-drift, and 98.61% for random under 5-folds. The reason for this can be attributed to the fact that it leveraged both of the strengths of the other models.

An added benefit of binary transformation approach is since it trains a separate classifier for each class, one can assess the performance of each classifier. Below, the performance assessment of each classifier is shown. The Bias classifier refers to the classifier trained to identify the bias fault class and so on.

Table 3 show the performance result of the MLP, CNN and CNN-MLP model evaluated using 5 folds cross-validation. Accordingly, it can be observed that the model was able to identify drift, poly-drift and random faults remarkably, but performed less than desired for the bias set.

Table 4 show the performance results of CNN, MLP and CNN-MLP model evaluated against 10 folds cross-validation respectively.

Accordingly, the CNN-MLP model outperformed both the other models. Such better performance achievement is due to, the combination of CNN and MLP that is allowed the model to construct a more effective representation of the data, capturing both the spatial and temporal patterns of the data.

Confusion matrix visualization: the performance of each classifier for the three models using the confusion matrix is shown in the below description.

Figure 7 illustrate the confusion matrices for the bias, drift, random, Polydrift and malfunction set experimented with MLP model. For example, for the bias set, from a total of 59,515 samples, the model was able to correctly classify 25,836 as normal and 27,645 as bias. The model incorrectly classified 3,977 normal samples as bias and 2057 bias samples as normal. For the drift set, from a total of 60,146 samples, the model was able to correctly classify 30,062 as normal and 29,574 as drift, and incorrectly classified 68 normal samples as drift and 442 drift samples as normal. For the random set, out of total of 70,137, the model classified correctly all the normal samples and 33,965 random samples correctly. It incorrectly classified 955 random values as normal. For the Polydrift set, out of total of 60,137, 30,072 normal samples were correctly classified with only 55 samples incorrectly classified as Polydrift. 29,314 Polydrift values were correctly classified with only 696 samples incorrectly classified as normal.

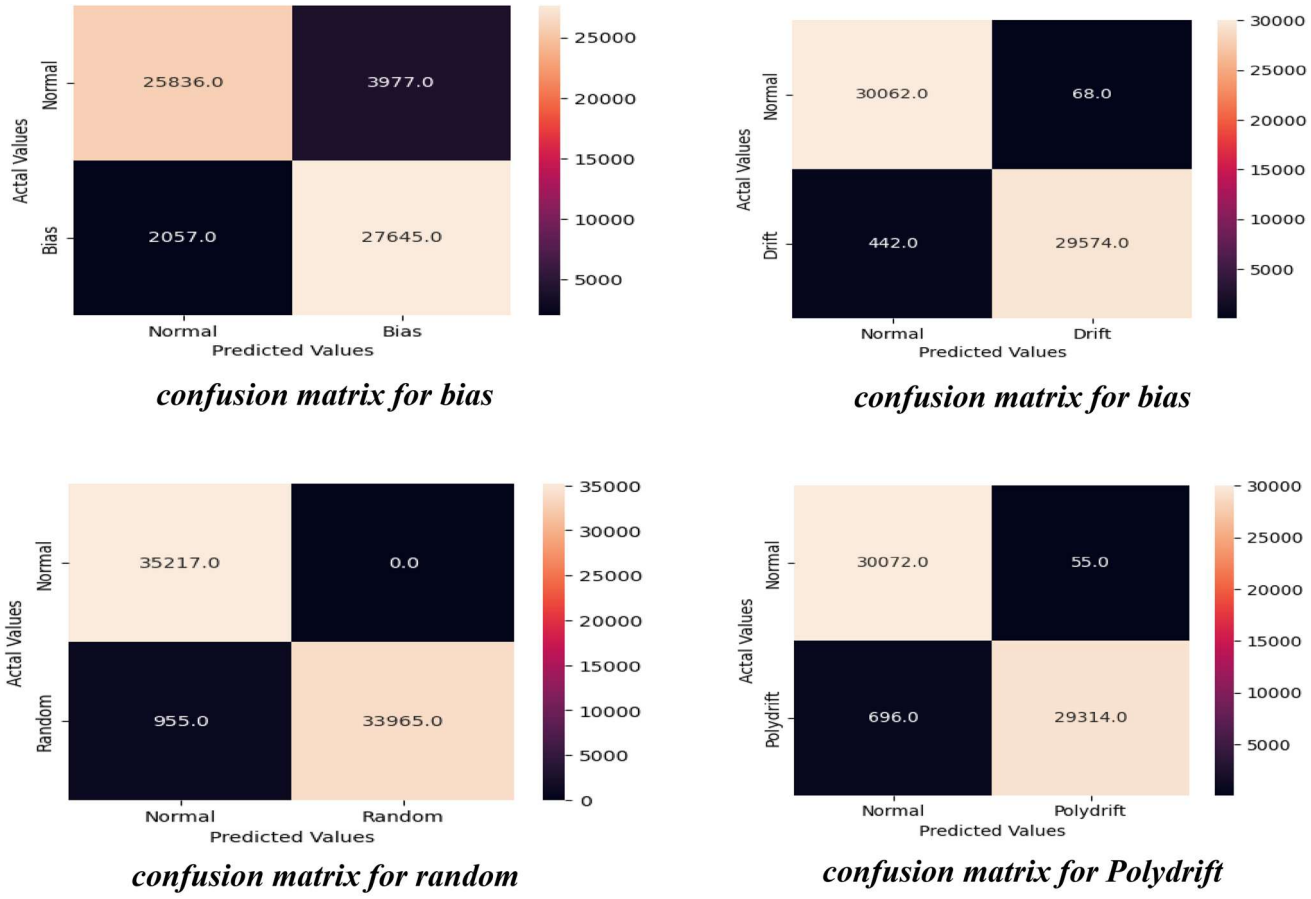


Fig. 7 Confusion matrix MLP

Figure 8 illustrate the performance of CNN model using confusion matrices. For example, for the bias set, out of a total of the 59,515 samples, 25,919 were correctly classified as normal and 28,329 were correctly classified as bias. 3,894 normal samples were incorrectly classified as bias and 1,373 bias samples were incorrectly classified as normal. For the drift set, out of the total 60,146 samples, 30,052 were correctly classified as normal and 29,685 were correctly classified as drift. 78 normal samples were incorrectly classified as drift and 331 drift samples incorrectly classified as normal.

Figure 9 shows the performance of the CNN-MLP model using the confusion matrices. Accordingly, for the bias set, out of the total 59,515 samples, 26,532 were correctly classified as normal samples and 27,606 correctly classified as bias samples. 3281 normal samples were incorrectly classified as bias and 2096 bias samples classified as normal. For the drift set, out of the total 60,146 samples all normal samples were correctly classified and 29,511 drift samples were correctly classified. 505 drift samples were incorrectly classified as normal. For the random set, out of the total 70,137 samples, all normal samples were correctly classified and 33,966 samples were correctly classified as random. 954 random samples were incorrectly classified as normal.

5 CNN-MLP classifiers learning curve

Figures 10, 11, 12, 13, 14, 15, 16, 17 show the plot of the accuracy and loss during training and validation for the CNN-MLP classifiers. The classifiers were trained using an EarlyStopping of 10 patience to prevent underfitting or overfitting.

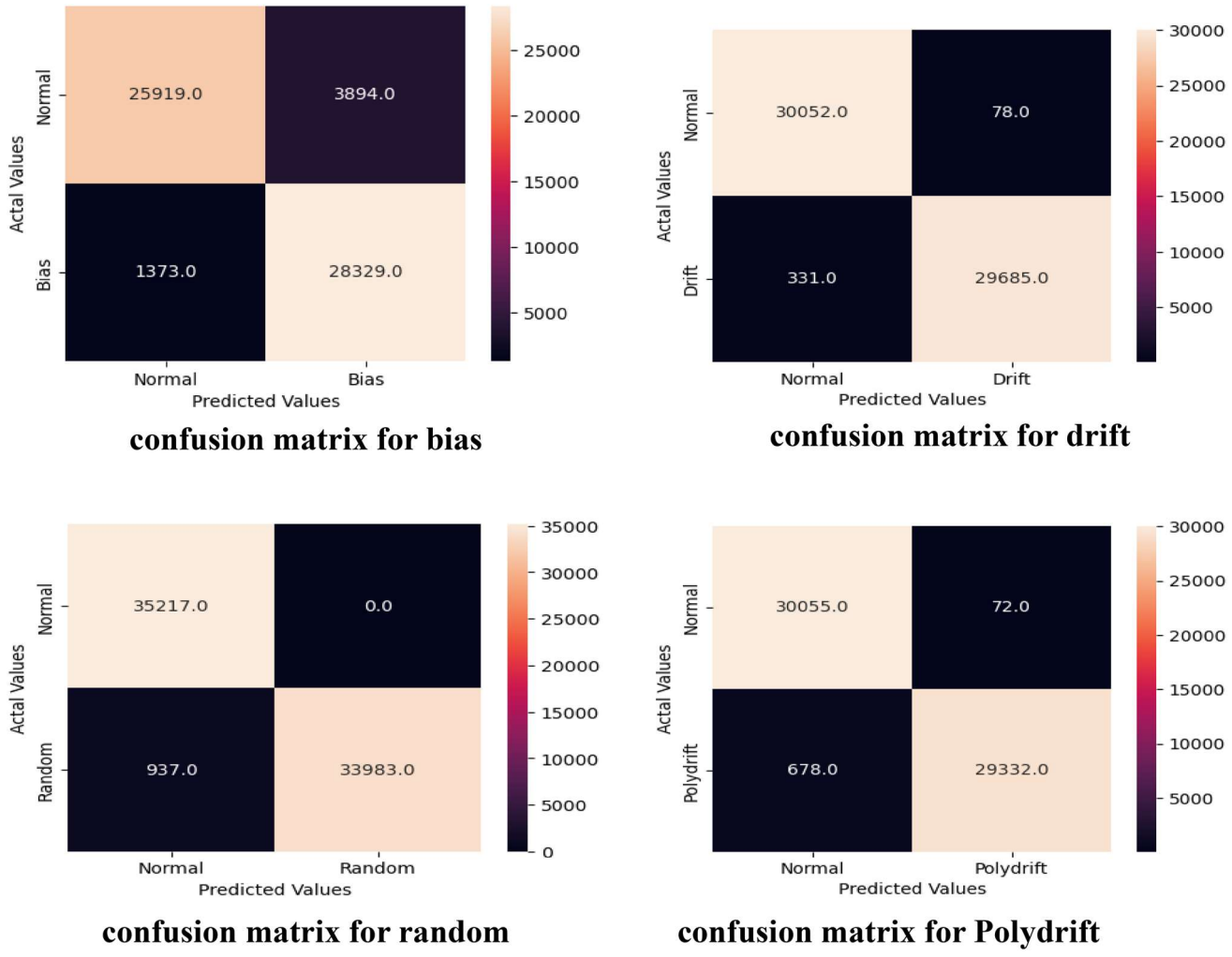


Fig. 8 Confusion matrix for CNN

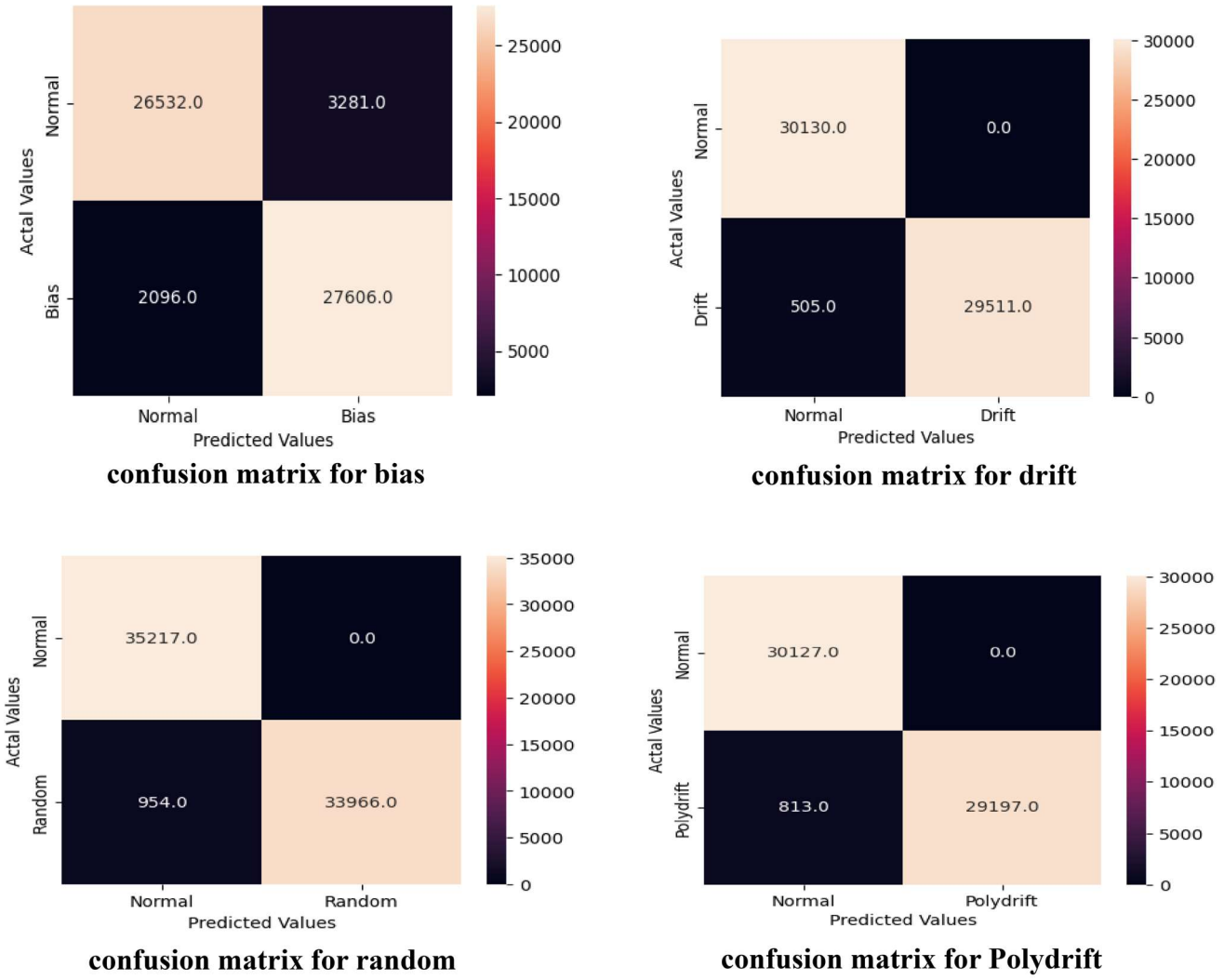


Fig. 9 Confusion matrix CNN-MLP

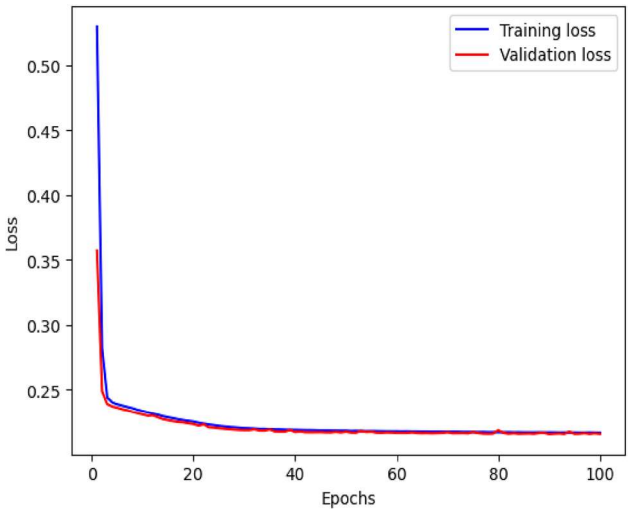


Fig. 10 Training and validation loss for bias fault

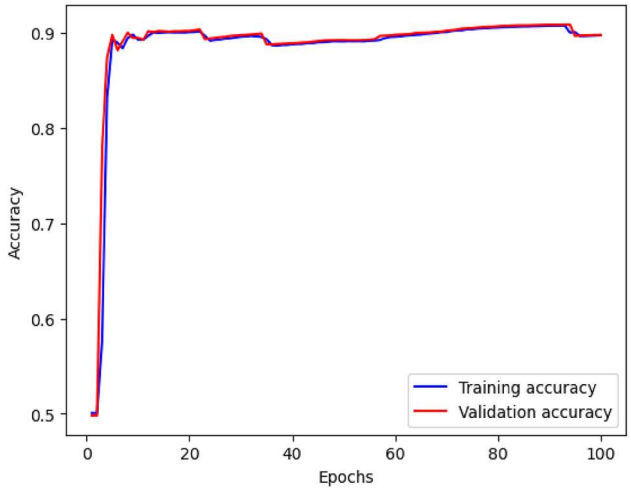


Fig. 11 Training and validation accuracy for bias fault

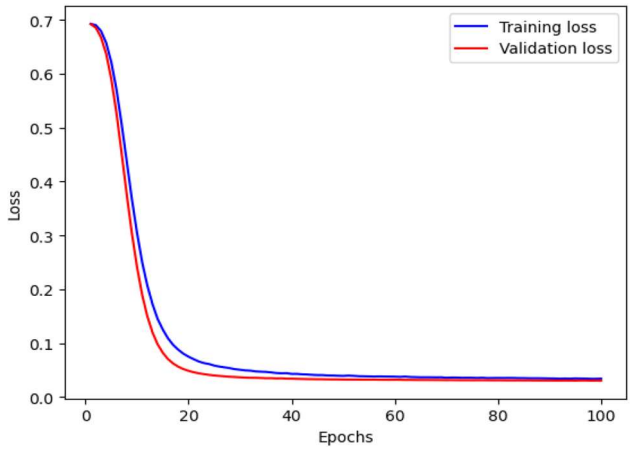


Fig. 12 Training and validation loss for drift fault

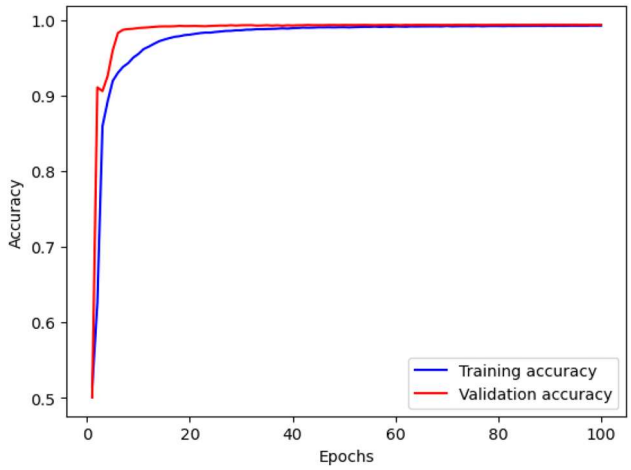


Fig. 13 Training and validation accuracy for drift fault

6 Summary of individual classifiers

Table 5 shows the performance of each individual binary classifiers for the multi-class classification problem. In this work, the faults names as the classifier's names have been used for simplicity, i.e., Bias classifier refers to the classifier that is trained to identify the bias fault class and so on.

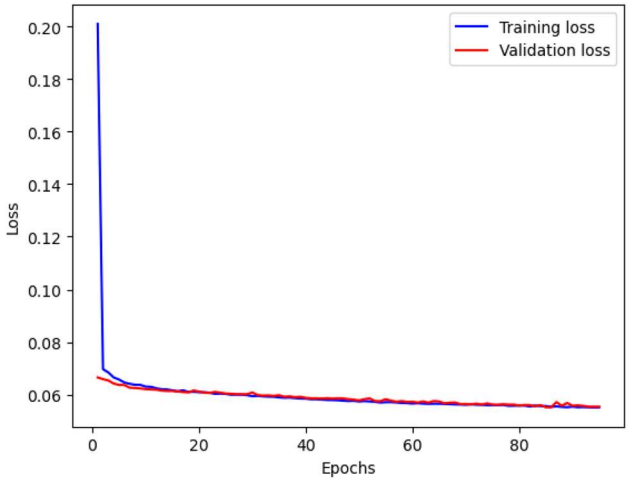


Fig. 14 Training and validation loss for random fault

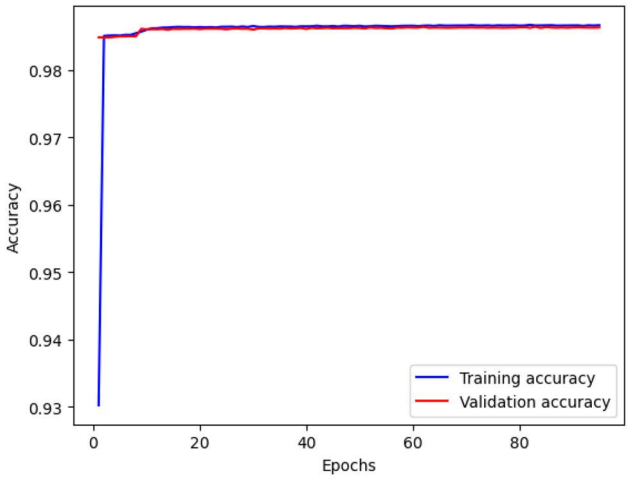


Fig. 15 Training and validation accuracy for random fault

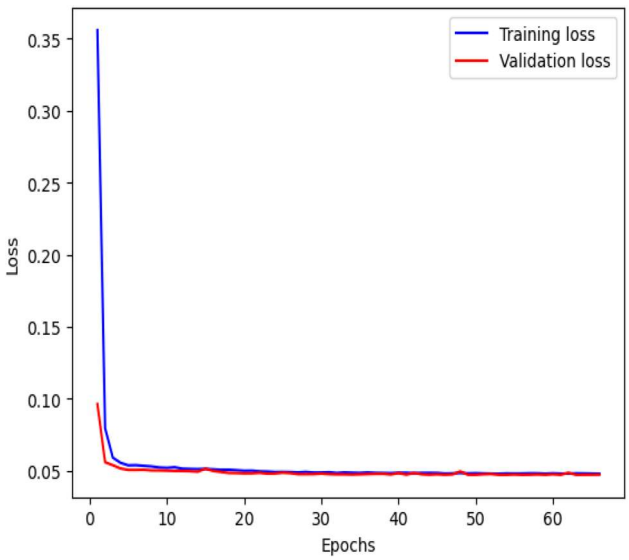


Fig. 16 Training and validation loss for poly-drift fault

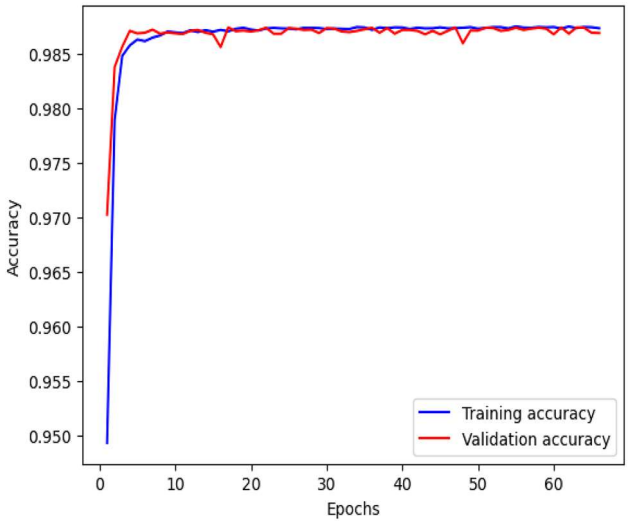


Fig. 17 Training and validation accuracy for poly-drift fault

Table 5 Performance of individual binary classifiers

Classifiers	Model	Accuracy	Precision	Recall	F1
Bias	MLP	94.78	94.95	94.78	94.77
	CNN	96.05	96.16	96.10	96.10
	CNN-MLP	96.11	96.42	96.12	96.10
Drift	MLP	99.03	99.17	99.16	99.16
	CNN	99.13	99.30	99.29	99.29
	CNN-MLP	99.33	99.33	99.33	99.33
Random	MLP	98.50	98.49	98.48	98.48
	CNN	98.54	98.58	98.54	98.58
	CNN-MLP	98.61	98.65	98.61	98.61
Poly-drift	MLP	98.81	98.83	98.81	98.81
	CNN	98.63	98.63	98.64	98.64
	CNN-MLP	98.81	98.83	98.81	98.81

Table 6 Macro-average performance of classification models

Models	Accuracy	Precision	Recall	F1
MLP	97.78	97.86	97.80	97.80
CNN	98.08	98.16	98.14	98.15
CNN-MLP	98.215	98.30	98.21	98.21

In multi-class classification, there are 3 main techniques for assessing the overall performance of a model across all the classes. This are macro-average, micro-average and weighted average. The overall performances of the models are discussed using the macro-average technique because this technique gives equal importance to all classes.

Table 6 shows the overall performance of the models using the macro-average technique. This technique is chosen because it gives equal importance to all classes.

For a model comparison, we have considered the work in [41] as a baseline in order to compare the performance of our proposed CNN-MLP. The baseline work evaluated the model (CDHMM-LVQ) against 10%, 20%, 30%, 40% and 50% injection rate and performed the classification for the faults bias, drift, random and spike. Thus, we used the performance of the 20% rate to compare our model against since the injection rate for our dataset has been similar in this consideration. Accordingly, CDHMM-LVQ model achieved the performance of bias 98.33%, drift 96.67%, spike 96.67%, and random 100%, whereas our model achieved bias 96.11%, drift 99.33%, poly-drift 98.81% and random 98.61%. When we compute an average accuracy, our CNN-MLP model achieved 98.21% while the existing model achieved 97.91% accuracy. In this regard, our presented work improved an average accuracy of 0.3% over the previous work. In this regard, the achieved

results desire to capitalize on the benefits of IoT technology and harness the potential of these technologies to an even greater extent than those we are currently using, while at the same time trying to minimize the impact that results when the system malfunctions or fails.

7 Conclusions

In this study a proactive approach has been developed that predicts the potential occurrence of faults using hybrid deep learning models. The solution involves a two-stage procedure where in the first stage, a hybrid CNN-LSTM model is trained on a clean non-faulty historical data of the sensors to perform a multi-step forecasting of future values. In the second stage, a hybrid CNN-MLP model is trained on fault injected sensor data to learn the patterns of the different fault types and classify new data accordingly. The forecasted values of the first hybrid model are then passed as input to the second hybrid model to determine its status.

For the regression task, with different forecasting models like LSTM, GRU, BiLSTM, CNN-LSTM, CNN-BiLSTM and CNN-GRU have been experimented both before and after incorporating rolling window statistical features. Also, evaluated each of them using evaluation metrics RMSE, MSE and MAE and finally chose CNN-LSTM with incorporated rolling window features as it outperformed the other models with a MAE of 2.0957 for a 45 timesteps forecast. For the classification task, with MLP, CNN and CNN-MLP and evaluated them using the classification evaluation metrics accuracy, precision, recall and f1 using 5-folds and 10-folds cross validation and in both cases CNN-MLP outperformed the other models with accuracy of 96.11% for bias, 99.33% for drift, 98.61% for random and 98.81% for poly-drift, an average of 98.21% over the 4 faults. Finally, this wrok compared with the baseline work and achieved a 0.3% increase in the average accuracy across the 4 faults.

The outcomes of this study have implications for various sectors, including manufacturing, agriculture, transportation, and healthcare, where the prediction of potential sensor faults can prevent catastrophic failures, reduce maintenance costs, and optimize system performance. In all these and other cases, by integrating the model at different levels of IoT's architectures (edge, gateway, cloud), the IoT system can have the ability to take proactive measures, minimize downtime, optimize decision-making, and enhance overall system reliability and efficiency. Furthermore, the proposed hybrid deep learning models can serve as a foundation for future research in the field of predictive maintenance and fault diagnosis.

Acknowledgements The authors would like to take this opportunity to acknowledge Adama Science and Technology University and Dilla University for providing the necessary facilities to support this investigation

Authors contribution AMS was responsible for investigating and executing the entire objective of the work, using proper research methodology. KAG was responsible for the topic selection, supervising and executing the research as per scientific principles. PJR was responsible for mathematical analysis, programs verification, literature execution and compiling the data.

Data availability The raw sensor data used in this study is publicly available at http://db.csail.mit.edu/labdata/labdata.html. The annotated data sets used during the current study have also been made publicly available by the authors that performed the fault injection and annotation at https://www.dropbox.com/s/l98aln9g87caatk/datasets-release-v3.zip?dl=0.

Declarations

Competing interests The authors declare no competing interests.

Ethical approval All procedures performed in the studies were in accordance with the ethical standards of the institutional and/or national research committee and with the comparable ethical standards.

Informed consent For this type of study, formal consent is not required. Authors give consent for the publication of the submitted research article in Discover Applied Sciences.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Dogra R, Rani S, Gianini G. REERP: a region-based energy-efficient routing protocol for IoT wireless sensor networks. Energies. 2023;16(17):6248.
2. Aba EN, Olugboji OA, Nasir A, Olutoye MA, Adedipe O. Petroleum pipeline monitoring using an internet of things (IoT) platform. SN Appl Sci. 2021;3:1–12.
3. Sethi P, Sarangi SR. Internet of things: architectures, protocols, and applications. J Electr Comput Eng. 2017. https://doi.org/10.1155/2017/9324035.
4. Yang C, Shen W, Wang X. The internet of things in manufacturing: key issues and potential applications. IEEE Syst Man Cybern Mag. 2018;4(1):6–15. https://doi.org/10.1109/MSMC.2017.2702391.
5. Xu J, Gu B, Tian G. Review of agricultural IoT technology. Artif Intell Agric. 2022;6:10–22. https://doi.org/10.1016/J.AIIA.2022.01.001.
6. Muthuramalingam S, Bharathi A, Rakesh Kumar S, Gayathri N, Sathiyaraj R, Balamurugan B. Iot based intelligent transportation system (iot-its) for global perspective: a case study. Intell Syst Ref Libr. 2019;154:279–300. https://doi.org/10.1007/978-3-030-04203-5_13/COVER.
7. Xia F. Wireless sensor technologies and application. Sensors. 2009;9(11):8824–30. https://doi.org/10.3390/S91108824.
8. Zaidan AA, Zaidan BB, Qahtan MY, Albahri OS, Albahri AS, Alaa M, Jumaah FM, Talal M, Tan KL, Shir WL, Lim CK. A survey on communication components for IoT-based technologies in smart homes. Telecommun Syst. 2018;69(1):1–25. https://doi.org/10.1007/S11235-018-0430-8/FIGURES/8.
9. Gemeda KA, Gianini G, Libsie M. An evolutionary cluster-game approach for wireless sensor networks in non-collaborative settings. Pervas Mob Comput. 2017;42:209–25.
10. Titouna C, Naït-Abdesselam F, Khokhar A. DODS: a distributed outlier detection scheme for wireless sensor networks. Comput Netw. 2019;161:93–101. https://doi.org/10.1016/J.COMNET.2019.06.014.
11. Li D, Wang Y, Wang J, Wang C, Duan Y. Recent advances in sensor fault diagnosis: a review. Sens Actuators A. 2020;309:111990. https://doi.org/10.1016/J.SNA.2020.111990.
12. Manzoni E, Rampazzo M, Favero SD. Detection of glucose sensor faults in an artificial pancreas via whiteness test on Kalman filter residuals. IFAC-PapersOnLine. 2021;54(7):274–9. https://doi.org/10.1016/J.IFACOL.2021.08.371.
13. Welcer M, Szczepański C, Krawczyk M. The impact of sensor errors on flight stability. Aerospace. 2022;9(3):169. https://doi.org/10.3390/AEROSPACE9030169.
14. Ali F, Ali A, Imran M, Naqvi RA, Siddiqi MH, Kwak KS. Traffic accident detection and condition analysis based on social networking data. Accid Anal Prev. 2021;151:105973. https://doi.org/10.1016/J.AAP.2021.105973.
15. Gao Z, Cecati C, Ding SX. A survey of fault diagnosis and fault-tolerant techniques-part I: fault diagnosis with model-based and signal-based approaches. IEEE Trans Ind Electron. 2015;62(6):3757–67. https://doi.org/10.1109/TIE.2015.2417501.
16. Habibi H, Howard I, Simani S. Reliability improvement of wind turbine power generation using model-based fault detection and fault tolerant control: a review. Renew Energy. 2019;135:877–96. https://doi.org/10.1016/J.RENENE.2018.12.066.
17. Clouqueur T, Ercevik O, Saluja KK. Efficient signature-based fault diagnosis using variable size windows. In: VLSI Design 2001. Fourteenth International Conference on VLSI Design 2001 Jan 7 (pp. 391–396). IEEE. https://doi.org/10.1109/ICVD.2001.902690
18. Subbaraj P, Kannapiran B. Artificial neural network approach for fault detection in pneumatic valve in cooler water spray system. Int J Comput Appl. 2010;9(7):43–52. https://doi.org/10.5120/1395-1881.
19. Huang J, Li M, Zhang Y, Mu L, Ao Z, Gong H. Fault detection and classification for sensor faults of UAV by deep learning and time-frequency analysis. In: 2021 40th Chinese Control Conference (CCC) 2021 Jul 26 (pp. 4420–4424). IEEE. https://doi.org/10.23919/CCC52363.2021.9550141
20. Uppal M, Gupta D, Juneja S, Dhiman G, Kautish S. Cloud-based fault prediction using IoT in Office Automation for Improvisation of Health of Employees. J Healthcare Eng. 2021;2021:1–13. https://doi.org/10.1155/2021/8106467.
21. Jana D, Patil J, Herkal S, Nagarajaiah S, Duenas-Osorio L. CNN and Convolutional Autoencoder (CAE) based real-time sensor fault detection, localization, and correction. Mech Syst Signal Process. 2022;169:108723. https://doi.org/10.1016/J.YMSSP.2021.108723.
22. Djeziri MA, Benmoussa S, Sanchez R. Hybrid method for remaining useful life prediction in wind turbine systems. Renew Energy. 2018;116:173–87. https://doi.org/10.1016/J.RENENE.2017.05.020.
23. Gebremeskel TG, Gemeda KA, Krishna TG, Ramulu PJ. DDoS attack detection and classification using hybrid model for multicontroller SDN. Wirel Commun Mob Comput. 2023. https://doi.org/10.1155/2023/9965945.
24. Ni K, Ramanathan N, Chehade MNH, Balzano L, Nair S, Zahedi S, Kohler E, Pottie G, Hansen M, Srivastava M. Sensor network data fault types. ACM Trans Sens Netw. 2009;5(3):1–29. https://doi.org/10.1145/1525856.1525863.
25. Yin S, Wang G. An approach for robust data-driven fault detection with industrial application. IECON Proceedings (Industrial Electronics Conference), 3317–3322; 2013. https://doi.org/10.1109/IECON.2013.6699660
26. Srimani, S., Ghosh, K., & Rahaman, H. (2016). Parametric Fault Detection in Analog Circuits: A Statistical Approach. Proceedings of the Asian Test Symposium, 275–280. https://doi.org/10.1109/ATS.2016.55
27. Rahimilarki, R., Gao, Z., Jin, N., Binns, R., & Zhang, A. (2020). Data-driven Sensor Fault Estimation for the Wind Turbine Systems. IEEE International Symposium on Industrial Electronics, 2020-June, 1211–1216. https://doi.org/10.1109/ISIE45063.2020.9152490
28. Schneider P, Xhafa F.. Anomaly detection: concepts and methods. anomaly detection and complex event processing over IoT data streams, 49–66; 2022. https://doi.org/10.1016/B978-0-12-823818-9.00013-4
29. Jihani N, Kabbaj MN, Benbrahim M. Sensor fault detection and isolation for smart irrigation wireless sensor network based on parity space. Int J Electr Comput Eng. 2023;13(2):1463–71. https://doi.org/10.11591/IJECE.V13I2.PP1463-1471.
30. Hashimoto M, Kawashima H, Oba F. A multi-model based fault detection and diagnosis of internal sensor for mobile robot. IEEE Int Conf Intell Robots Syst. 2003;4:3787–92. https://doi.org/10.1109/IROS.2003.1249744.
31. He C, Zhang X, Jia B. UIO based robust fault diagnosis approach for aero-engine fiber-optic sensor. IEEE International Conference on Automation Science and Engineering, 550–553; 2013. https://doi.org/10.1109/COASE.2013.6653947

32. Yan X, Guan T, Fan K, Sun Q. Novel double layer BiLSTM minor soft fault detection for sensors in air-conditioning system with KPCA reducing dimensions. J Build Eng. 2021;44:102950. https://doi.org/10.1016/J.JOBE.2021.102950.
33. Alwan AA, Brimicombe AJ, Ciupala MA, Ghorashi SA, Baravalle A, Falcarin P. Time-series clustering for sensor fault detection in large-scale cyber-physical systems. Comput Netw. 2022;218:109384. https://doi.org/10.1016/J.COMNET.2022.109384.
34. Zhao W, Luo H, Liu Q, Ji H, Sheng N. Incipient sensor fault detection by directly monitoring sliding window based singular values. IFAC-PapersOnLine. 2022;55(6):637–42. https://doi.org/10.1016/J.IFACOL.2022.07.199.
35. Uppal M, Gupta D, Juneja S, Sulaiman A, Rajab K, Rajab A, Elmagzoub MA, Shaikh A. Cloud-based fault prediction for real-time monitoring of sensor data in hospital environment using machine learning. Sustainability. 2022;14(18):11667. https://doi.org/10.3390/SU141811667.
36. Liu WX, Yin RP, Zhu PY. Deep learning approach for sensor data prediction and sensor fault diagnosis in wind turbine blade. IEEE Access. 2022;10:117225–34. https://doi.org/10.1109/ACCESS.2022.3219480.
37. Wahid A, Breslin JG, Intizar MA. Prediction of machine failure in industry 4.0: a hybrid CNN-LSTM framework. Appl Sci. 2022;12(9):4221. https://doi.org/10.3390/APP12094221.
38. Safavi S, Safavi MA, Hamid H, Fallah S. Multi-sensor fault detection, identification, isolation and health forecasting for autonomous vehicles. Sensors. 2021. https://doi.org/10.3390/S21072547.
39. Biddle L, Fallah S. A novel fault detection, identification and prediction approach for autonomous vehicle controllers using SVM. Autom Innov. 2021;4(3):301–14. https://doi.org/10.1007/S42154-021-00138-0/FIGURES/8.
40. Wang ZM, Song GH, Gao C. An isolation-based distributed outlier detection framework using nearest neighbor ensembles for wireless sensor networks. IEEE Access. 2019;7:96319–33. https://doi.org/10.1109/ACCESS.2019.2929581.
41. Emperuman M, Chandrasekaran S. Hybrid continuous density hmm-based ensemble neural networks for sensor fault detection and classification in wireless sensor network. Sensors (Switzerland). 2020. https://doi.org/10.3390/S20030745.
42. De Bruijn B, Nguyen TA, Bucur D, Tei K. Benchmark datasets for fault detection and classification in sensor data. SENSORNETS 2016 - Proceedings of the 5th International Confererence on Sensor Networks, 185–195; 2016. https://doi.org/10.5220/0005637901850195
43. https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/sequence/TimeseriesGenerator
44. Mishra AK, Mohanty SK. Intermittent Fault Diagnosis in Sensors using Artificial Neural Network: A Comparative Study. 2022 IEEE International Conference on Current Development in Engineering and Technology (CCET), 1–7; 2022. https://doi.org/10.1109/CCET56606.2022.10080532
45. Marcellino M, Stock JH, Watson MW. A comparison of direct and iterated multistep AR methods for forecasting macroeconomic time series. J Econometrics. 2006;135(1–2):499–526. https://doi.org/10.1016/J.JECONOM.2005.07.020.
46. Taieb SB, Hyndman RJ. Recursive and direct multi-step forecasting: the best of both worlds; 2022. https://doi.org/10.26180/21500169.V1
47. Wibawa AP, Utama ABP, Elmunsyah H, Pujianto U, Dwiyanto FA, Hernandez L. Time-series analysis with smoothed convolutional neural network. J Big Data. 2022;9(1):1–18. https://doi.org/10.1186/S40537-022-00599-Y/TABLES/12.
48. Yamashita R, Nishio M, Do RKG, Togashi K. Convolutional neural networks: an overview and application in radiology. Insights Imaging. 2018;9(4):611–29. https://doi.org/10.1007/S13244-018-0639-9.
49. Kim S, Kim H. A new metric of absolute percentage error for intermittent demand forecasts. Int J Forecast. 2016;32(3):669–79. https://doi.org/10.1016/J.IJFORECAST.2015.12.003.
50. Bouktif S, Fiaz A, Ouni A, Serhani MA. Optimal deep learning LSTM model for electric load forecasting using feature selection and genetic algorithm: comparison with machine learning approaches. Energies. 2018;11(7):1636. https://doi.org/10.3390/EN11071636.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.